

Application design: Car Rental System

1. Overview

A web-based Car Rental System that allows customers to book vehicles and employees to manage fleet, rentals, and invoices. The system uses a REST API with token-based authentication and a simple web interface.

Main user roles: Customer, Employee, Administrator.

2. Architecture

Components:

- Frontend: Web UI (React or HTML/JS) for login, browsing, and reservations.
- Backend API: (Node/Express, Django REST, or Spring Boot) exposes endpoints for authentication, CRUD operations, and generating invoices, car rental agreements.
- Database: PostgreSQL for storing users, vehicles, rentals, and payments.

3. Key Features

- **Auth:** Register/Login using JWT tokens.
- **CRUD:** Manage Vehicles, Customers, Employees.
- **Reservations:** Create and confirm bookings. (REST API)
- **Rentals:** Track active/closed rentals with fuel and mileage (REST API).
- **Reports:** Generate rental agreements and invoices (REST API + PDF library).

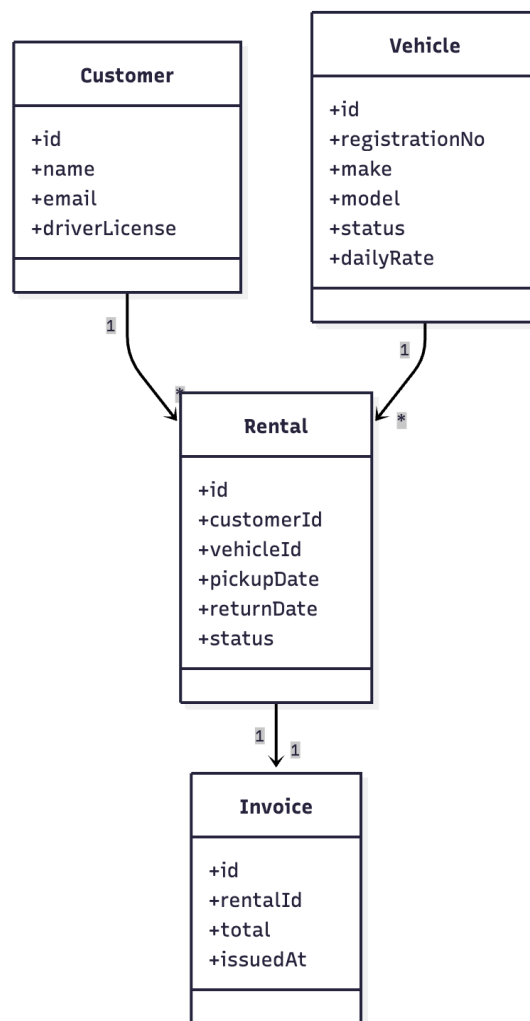
4. Core Entities

Entity	Attributes	Relationships
Vehicle	id, regNo, make, model, status, dailyRate	1..* Rental
Customer	id, name, email, licenseNo	1..* Rental
Employee	id, name, role	—
Rental	id, customerId, vehicleId, pickupDate, returnDate, status	belongs to Customer & Vehicle
Invoice	id, rentalId, totalAmount, issuedAt	1–1 Rental

5. HTTP API Description

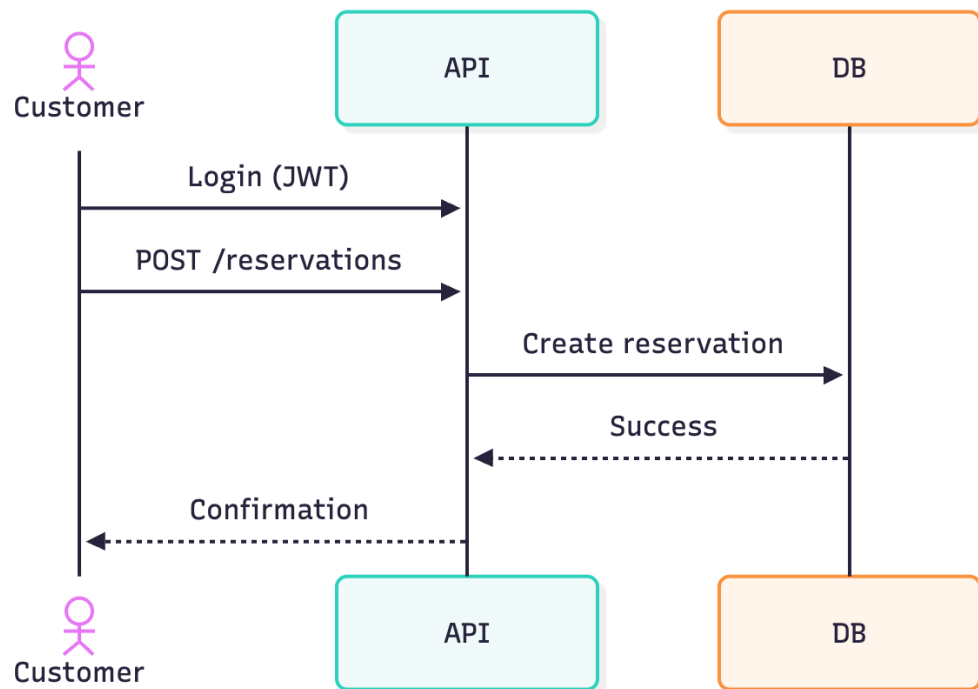
Endpoint	HTTP Method	Description
/auth/register	POST	Register new customer
/auth/login	POST	Obtain JWT token
/vehicles	GET/POST/PUT/DELETE	Manage vehicles
/reservations	POST	Create reservation
/rentals	POST/PUT	Manage rentals
/reports/invoice/{id}	GET	Generate invoice

6. UML Class Diagram



* Class diagram created using [Mermaid.js](#)

7. Example Flow Diagram (Reservation)



* Flow diagram created using [Mermaid.js](https://mermaid.js.org/)